

Connect WooCommerce ↔ Skipily (step by step)

Prerequisite: First read `00-Skipily-Integration-Basics.md` and have your **Skipily API key** ready. WooCommerce is a plugin for WordPress – so you need a WordPress login with admin rights.

We connect both via the free no-code tool **Make.com**. You click everything together – no programming.

 **Shortcut – ready-made template:** A finished Make template "WooCommerce ↔ Skipily" does not exist in the public gallery (yet), but we provide **importable blueprints** (folder `blueprints/`) that pre-fill the entire Skipily part (URL, headers, JSON): `skipily-create-product.json` (Part A), `skipily-receive-order.json` (Part B), `skipily-report-shipment.json` (Part C). **Import:** New scenario → bottom "..."
(More) → **"Import Blueprint"** → choose file. After that you only need to insert your `x-api-key`, add the WooCommerce module and map the fields – exactly what Parts A–C show.

What works in the end:

- New/changed product in WooCommerce → automatically appears on Skipily.
- Order on Skipily → automatically lands in WooCommerce.
- Shipment recorded in WooCommerce → status + tracking number go back to Skipily.

Allow **about 45–60 minutes** for the initial setup.

Part 0 – Preparation (10 min)

1. **Create a Make.com account:** <https://www.make.com> → "Get started free".
2. **Create a WooCommerce API key** (so Make may read/write your shop):
 - WordPress admin → **WooCommerce** → **Settings** → **Advanced** → **REST API**.
 - Click **"Add key"**.
 - Description: `Skipily`, User: yourself, **Permissions: Read/Write**.
 - Click **"Generate API key"** → a **Consumer Key** and a **Consumer Secret** appear. **Copy and store them safely now** (the secret is shown only once!).
3. Have your **Skipily API key** from the provider portal ready (see Basics).

Now you have three credentials: WooCommerce key, WooCommerce secret, Skipily API key.

Part A – Send products from WooCommerce to Skipily (15 min)

Goal: When you create a product in WooCommerce, it is automatically reported to Skipily.

1. In **Make.com** click **"Create a new scenario"** (top right).
2. Click the large **"+"** → search for **"WooCommerce"** → choose the module **"Watch Products"**.
 - The first time, Make asks for a **"Connection"**: click **"Add"** and enter your shop address (`https://your-shop.com`), **Consumer Key** and **Consumer Secret** from Part 0 → **Save**.
 - For "Watch" choose **"Created and updated"**.
3. Next to WooCommerce click **"+"** again → **"HTTP"** → module **"Make a request"**. This is the call to the Skipily API.
4. Fill in this HTTP module as follows:
 - **URL:** `https://vcjwlyqkfkszumdrfvtn.supabase.co/functions/v1/products-api`
 - **Method:** `POST`
 - **Headers** (click "Add item", one header per row):
 - Name `Authorization` – value `Bearer eyJhbGciOi...` (long public value from the Basics)
 - Name `x-api-key` – value = **your Skipily API key**
 - Name `Content-Type` – value `application/json`
 - **Body type:** `Raw` → **Content type:** `JSON (application/json)`
 - **Request content:** here you build the product data. Click into the fields and pick the matching WooCommerce fields from the WooCommerce module above (the coloured "chips"). Template:

```
{
  "name": "{{Product name from WooCommerce}}",
  "sku": "{{SKU from WooCommerce}}",
  "price": "{{Price from WooCommerce}}",
  "currency": "EUR",
  "stock_quantity": "{{Stock quantity from WooCommerce}}",
  "in_stock": true,
  "description": "{{Short description from WooCommerce}}"
}
```

5. **Save** (disk icon at the bottom). Test with **"Run once"**: create a test product in WooCommerce → the scenario should run green and the product appears on Skipily.
6. Bottom left, turn the **"Scheduling"** switch **ON** (e.g. every 15 min) so it runs automatically for good.

Tip – avoid duplicates: Store the `id` returned by Skipily for each product (e.g. in a WooCommerce "meta field"). For later changes, use **PUT** on `.../products-api?id=THE-ID` instead of creating a new one every time. For the start, simple creation is enough.

Part B – Create Skipily orders in WooCommerce (15 min)

Goal: When someone buys via Skipily, the order should appear automatically in WooCommerce. For this, Skipily "rings" a web address that Make gives you (**webhook**).

1. Create a **new scenario** in Make.com.
2. Choose the first module **"Webhooks"** → **"Custom webhook"** → **"Add"** → name e.g. `Skipily order s` → Make shows you an **address (URL)** → **copy it**.
3. Store this URL in Skipily: **Provider portal** → **Settings** → **Integrations** → **"Webhook URL for orders"** → paste → save. (No field found? Send the URL to support@skipily.app and we'll enter it.)
4. Back in Make: **"+"** → **"WooCommerce"** → **"Create an Order"**. Map the fields from the Skipily webhook (customer name, items, quantity, price, address).
5. **Save** and **Scheduling ON**.
6. **Test:** trigger a test order on Skipily (or ask us to) → the order should appear in WooCommerce.

Remember the Skipily order ID from the webhook (field `order_id` or `order_number`) – you need it in Part C for the shipment feedback. Best to write it into a WooCommerce order note/meta field.

Part C – Report shipment & tracking back to Skipily (10 min)

Goal: As soon as you ship in WooCommerce, the customer sees "shipped" + tracking number on Skipily.

1. Create a **new scenario** in Make.com.
2. First module **"WooCommerce"** → **"Watch Orders"**, for "Watch" **"Updated"**, and in the filters only let through orders with status **"completed/shipped"** (filter between the modules: gear/wrench → condition "Status = completed").
3. Second module **"HTTP"** → **"Make a request"**:
 - **URL:** `https://vcjwlyqkfkszumdrfvtn.supabase.co/functions/v1/orders-api?id={{Skipily order ID}}` (the ID from Part B – pull it from the order meta field)
 - **Method:** `PUT`
 - **Headers:** the same three as in Part A (`Authorization`, `x-api-key`, `Content-Type`).

- **Body → Raw → JSON:**

```
{
  "status": "shipped",
  "tracking_number": "{{Tracking number from WooCommerce}}",
  "tracking_url": "{{Tracking link, if available}}"
}
```

4. **Save, Run once** to test (set an order to "completed"), then **Scheduling ON**.

Done! Quick check

- ☐ Test product in WooCommerce → appears on Skipily (Part A)
- ☐ Test order on Skipily → appears in WooCommerce (Part B)
- ☐ Order set to "shipped" → Skipily shows "shipped" + tracking (Part C)

If something goes wrong

- **"401/403" in Make:** A header is wrong – check `x-api-key` (your Skipily key) and the `Authorization` value (copy exactly, no spaces).
- **"400 – name required":** The `name` field is missing in the product body. Re-map it.
- **Nothing happens:** Is the **Scheduling switch ON**? Is the scenario "active" (green) in Make on the left?
- **Help:** support@skipily.app – happy to help, ideally with a screenshot of the Make error message.